

Term Project: Handover Analysis in 5G Networks

성균관대학교 SKKU

SWE3022-41: Computer Networks

Andy Nguyen 앤디 (#2025319249)

2025년 10월 15일

Problem Statement

With the rapid advancement of 5G technology, understanding the factors that influence network performance under varying user conditions has become increasingly critical. When multiple User Equipment (UE) devices engage in simultaneous data transmissions, network resources are shared dynamically, potentially leading to congestion and performance degradation. My project investigates how the performance of a 5G network is affected by variations in the number of active UEs and traffic load parameters, specifically packet size and transmission interval. The objective is to analyze the relationship between these factors and key performance metrics to provide insights into the scalability and efficiency of 5G networks under diverse operating conditions.

Objective

The primary objective of this project is to design and implement a simulation program to analyze the performance of a 5G network under varying user and traffic conditions. The study aims to investigate how network performance scales with different numbers of User Equipment (UE) devices and payload sizes within a multicell 5G environment.

- **To analyze performance variations** in multicell environments based on different network parameters, including payload size and UE density.
- **To examine how network performance changes** with varying numbers of UEs and different UE movement patterns, reflecting realistic mobility scenarios.
- **To critically evaluate the effects of handovers** in dual connectivity scenarios and assess their impact on overall network performance.

Through these analyses, the project seeks to provide a comprehensive understanding of the scalability, reliability, and efficiency of 5G networks under diverse operational conditions.

[Type here]

Software Setup Information

System Requirements

For Windows, I use the MinGW64 Compiler Toolchain. The specific required software applications are OMNeT++ (v6.2.0), INET (v4.5.4), and Simu5G (v1.4.0), which will be used to conduct this project.

- **OMNeT++ (v6.2.0)**: discrete event simulator used as the base simulation engine.
- **INET Framework (v4.5.4)**: provided implementations of internet protocols such as UDP, TCP, and QUIC, as well as network applications, routing, and mobility models.
- **Simu5G (Version 1.4.0)**: served as the 5G network simulation module, enabling modelling of gNBs, UEs, and 5G Core components with realistic radio access and mobility management features.

All simulations were executed in the **gcc-release** mode for optimal runtime performance. The network topologies and parameters were configured through .ned and .ini files.

Project Conditions

Network Topology

The simulated network consisted of three gNBs (base stations) connected to a single 5G Core Network (5GC). Multiple User Equipment (UE) nodes were distributed across the coverage area of the cells. The multicell design allowed users to move between cells, thereby triggering handover events when coverage boundaries were crossed. Each UE was configured to download data from a remote server simultaneously, representing high network utilization scenarios. The simulation was executed for UE counts ranging from 10 to 100, in increments of 10.

Mobility Model

To emulate realistic user movement patterns:

- UEs used the Random Waypoint Mobility model from INET.
- Mobility speeds ranged from 1 m/s to 3 m/s (similar to pedestrian movement).
- For handover analysis, a subset of experiments incorporated vehicular mobility (10–20 m/s) to observe faster handover dynamics.

Mobility enabled the evaluation of both static and mobile user scenarios in multicell environments.

[Type here]

Communication Protocols

Three different transport-layer protocols were tested to compare data delivery efficiency:

1. **UDP (User Datagram Protocol):** baseline connectionless transmission model.
2. **TCP/IP (Transmission Control / Internet Protocol):** connection-oriented model with congestion control.
3. **QUIC (HTTP/3-based transport):** modern UDP-based transport offering multiplexed connections.

Each protocol was configured to transfer application-level data packets within the same network setup for fair comparison.

Traffic Configuration

All UEs were configured to simultaneously download data from a single remote server.

- **Packet payload size:** 100–1000 Bytes (increments of 100B).
- **Transmission interval:** constant rate for all UEs.
- **Application type:** file/video download sessions using UDP, TCP, or QUIC.

This configuration was designed to examine the impact of traffic load and packet size on overall network performance.

Performance/Data Analysis Metrics

The following key metrics were recorded for performance analysis:

- **Throughput (bps):** average rate of successful data delivery.
- **Packet Delay (ms):** average one-way delay between sender and receiver.
- **Inter-Packet Gap (ms):** time difference between successive packet receptions.
- **Packet Loss Ratio (%):** ratio of lost packets to total packets sent.

These parameters were obtained from .sca and .vec result files generated by OMNeT++ and were later processed using Python for analysis and visualization.

Handover & Dual Connectivity

Handover events between cells were monitored through Simu5G's internal 5G NR mobility management modules. Dual connectivity was optionally enabled in certain simulations to evaluate its impact on reducing delay and packet loss during handovers.

[Type here]

Implementation

Final Implementation Process

These are the phases of my implementation for this project:

Phase 1: Establish the Simulation Environment

Objective

Set up and verify the simulation environment with all necessary frameworks for 5G network experimentation.

Implementation

- Installed and configured **OMNeT++ 6.2**, **INET 4.5**, and **Simu5G 1.4.0** on **Windows 11**.
- Verified compatibility and successful integration among the three frameworks.
- Executed baseline tests using sample Simu5G scenarios to confirm functionality of the 5G New Radio (NR) modules and the EPC/5GC stack.

Outcome

A fully functional and verified simulation environment capable of executing customized 5G handover and transport-layer experiments.

Phase 2: Define the Simulation Topology

Objective

Design and implement a **multicell 5G NR environment** supporting multiple User Equipments (UEs) to enable realistic handover scenarios.

Implementation

- Created a multicell topology with two gNBs and multiple mobile UEs using the Simu5G **MultiCell_Standalone** network.
- Configured cell positions, mobility patterns, and X2 interfaces for inter-gNB communication.
- Defined realistic physical layer and propagation parameters such as frequency, numerology, and CQI (Continuous Quality Improvement) feedback intervals.
- Enabled handover and dynamic cell association to emulate real 5G movement scenarios.

Outcome

[Type here]

A scalable multicell 5G NR simulation environment ready for testing UE mobility, traffic load, and handover dynamics.

Phase 3: Implement Transport-Layer Traffic Models (UDP, TCP/IP, QUIC)

Objective

Develop and integrate traffic applications using different transport-layer protocols to evaluate performance under varying conditions.

Implementation

- Implemented UDP, TCP/IP, and QUIC sender and receiver modules (**FileSenderUDP/TCP/IP/QUIC** and **FileReceiverUDP/TCP/IP/QUIC**) in C++
- Defined protocol-specific socket handling, sequencing, packet generation, and loss detection logic.
- Ensured data transmission, reception, and statistical logging for each protocol in separate result directories (**results/UDP/**, **results/TCP/IP/**, **results/QUIC/**).
- Verified packet flow correctness and successful end-to-end delivery for all three protocols.

Source Code Description

- **FileReceiverUDP:** implements a lightweight, connectionless packet receiver using the UDP transport protocol. Because UDP provides no reliability, ordering, or congestion control, the module is responsible for fully tracking packet behavior at the application layer. It extracts the FilePacket header from each received datagram, measures one-way packet delay, computes inter-packet gaps, and detects packet loss by monitoring gaps in sequential frame IDs. It also maintains a sliding-window counter for computing instant and average throughput. All per-packet measurements are buffered and periodically written to CSV, followed by a final summary that includes delay, gap, throughput, and loss ratio. This module reveals the natural variability and potential unreliability of UDP under heavy load in 5G multicell environments.
- **FileReceiverTCP:** implements a stateful, reliable receiver using INET's TCP/IP suite socket abstraction. Because TCP/IP automatically handles retransmission, ordering, congestion control, and flow control, the receiver focuses on application-level metrics rather than low-level packet recovery. As TCP/IP delivers data in order, the receiver extracts the FilePacket payloads and records arrival delays, payload counts, and byte throughput without needing to detect losses manually (loss ratio is typically 0%). The module logs final aggregated statistics—average delay,

[Type here]

throughput, and frame counts—to a CSV output. This receiver represents a stable baseline for evaluating transport performance, reflecting the smooth, reliable nature of TCP/IP over the 5G network.

- **FileReceiverQUIC:** operates over UDP but emulates QUIC-style packet tracking by manually handling reliability-related metrics at the application layer. Each incoming packet's FilePacket header is parsed to extract timestamps, payload size, and frame ID, enabling the receiver to compute packet delay, inter-packet spacing, and instant throughput using a rolling time window. The module performs loss detection by checking for gaps in QUIC frame sequence numbers, similar to how QUIC detects packet loss in real deployments. It records per-packet measurements as well as final aggregated statistics, writing all results to a CSV file. This receiver demonstrates how QUIC's lightweight, UDP-based design can still provide stable and efficient performance even under dynamic 5G handover conditions.
- **FileSenderUDP:** implements a lightweight packet generator that sends FilePacket messages over UDP without reliability, ordering, or congestion control. At initialization, the module reads configuration parameters such as local/destination ports, payload size, and number of frames, then binds a UDP socket. A self-message timer periodically triggers `sendPacket()`, where a FilePacket containing the frame index, timestamp, and payload size is created and inserted into an OMNeT++ Packet object. The sender transmits each packet using `UdpSocket::sendTo()` toward the assigned UE. Because UDP provides no retransmission or flow control, this sender allows high throughput but exhibits possible packet loss and variable delay—behaviour that matches the empirical results of the simulation.
- **FileSenderTCP:** extends the TCP/IP socket callback interface to perform reliable, ordered frame delivery over TCP/IP. On initialization, the sender binds to a local port, resolves the destination UE address, and establishes a TCP/IP connection. Once the connection reaches the ESTABLISHED state, a periodic self-message triggers packet creation. For each frame, a `ByteCountChunk` representing the payload size is encapsulated into a Packet and delivered using `TcpSocket::send()`. TCP/IP ensures retransmission, congestion control, and byte-stream ordering, so this sender achieves extremely low loss and stable delay across all UE loads. The class also implements all TCP/IP callbacks (`socketEstablished`, `socketClosed`, `socketFailure`, etc.) to ensure safe operation and clean teardown, mirroring real TCP/IP stack behaviour.
- **FileSenderQUIC:** simulates QUIC-like behaviour over UDP by embedding sequencing and timestamp metadata inside each FilePacket. The module uses a UDP socket (not TCP/IP) but structures the packets to mimic QUIC's application-

[Type here]

layer reliability functions. During initialization, the sender resolves the UE's IP address, binds its socket, and schedules periodic transmission. On each timer event, it constructs a `FilePacket` that includes frame ID, total number of frames, payload size, and send timestamp, then inserts it into a `Packet` and sends it via `UdpSocket::sendTo()`. Unlike raw UDP, QUIC's metadata enables the receiver to detect loss, compute delay, calculate inter-packet gap, and emulate selective acknowledgment behaviour. Thus, this sender produces performance that falls between UDP and TCP/IP—higher throughput than TCP/IP while maintaining near-zero loss for typical loads.

- **FileTransfer.ned:** defines all application-layer modules used in the 5G multicell simulation, providing a unified NED interface for UDP, TCP/IP, QUIC, and general file-transfer applications. The file declares simple modules—`FileSenderUDP`, `FileReceiverUDP`, `FileSenderTCP`, `FileReceiverTCP`, `FileSenderQUIC`, `FileReceiverQUIC`, and their generic counterparts—each mapped to a corresponding C++ class via the `@class()` annotation. These NED definitions expose configurable parameters such as local/destination ports, payload size, start time, sampling interval, and number of frames, while also defining the standardized gate structure (`socketIn` and `socketOut`) that connects each application to the underlying transport or network stack. By separating module definitions from implementation, the file enables protocol interchangeability within `omnetpp.ini`, allowing the same simulation scenario to run UDP, TCP/IP, or QUIC traffic by simply selecting a different NED type. This modular design supports scalable experimentation across hundreds of UEs and ensures consistent integration with INET and Simu5G's node architecture.

Outcome

Three fully functional transport-layer traffic models (UDP, TCP/IP, and QUIC) implemented and integrated within the 5G handover simulation framework.

Phase 4: Configure Scalable Simulation Parameters

Objective

Automate experiments to evaluate protocol performance under different network loads and packet sizes.

Implementation

- Configured UE count to vary from 10 to 100 in increments of 10.
- Configured payload size to range from 100B to 1000B in increments of 100B.

[Type here]

- Implemented automatic naming conventions for result files to track runs by protocol, UE index, and repetition number.
- Applied randomized start times (`uniform(0.95s, 1.05s)`) to ensure asynchronous traffic generation.

omnetpp.ini Source Code Description

The **omnetpp.ini** file defines the complete runtime configuration for the 5G multicell handover simulation, including global parameters, mobility settings, PHY/MAC behavior, IPv4 configuration, and transport-layer protocol selection. It establishes a consistent simulation environment across UDP, TCP/IP, and QUIC scenarios by specifying simulation limits, warm-up periods, result directories, and memory-safe TCP settings. Each experiment—**Multicell-Handover-UDP**, **Multicell-Handover-TCP/IP**, and **Multicell-Handover-QUIC**—is declared as a separate configuration extending the base **MultiCell** profile, allowing the same 5G network, mobility patterns, carrier aggregation, and dual-cell layout to be reused across different transport-layer protocols. For each protocol, the file assigns the appropriate sender and receiver modules, sets UE positions and movement paths, parameterizes payload size, UE count, start times, and ensures that every simulation run generates separate scalar/vector files for automated performance evaluation. This configuration file serves as the central control point that binds the 5G topology, mobility model, application modules, and metrics collection into a coherent, reproducible experimental framework.

Outcome

A parameterized and scalable simulation matrix capable of batch-running multiple UE and payload scenarios automatically.

Phase 5: Enable Performance Metric Collection

Objective

Collect and record key Quality of Service (QoS) metrics for each protocol and test configuration.

Implementation

- Designed CSV output for all receiver modules (`_stats.csv`) capturing per-packet metrics such as:
 - Delay (s)
 - Inter-packet gap (s)
 - Instantaneous throughput (Mbps)
 - Packet loss ratio (%)

[Type here]

- Added automated summary generation at simulation termination (**finish()**), including averaged statistics per simulation run.

Outcome

Comprehensive raw performance data and summarized CSV results automatically generated for every test condition.

Phase 6: Analyse and Visualise Results

Objective

Process the recorded results to generate comparative performance analyses across protocols and network conditions.

Implementation

- Developed a Python-based analysis script (**analyze_results.py**) to parse the generated CSVs.
 - This is a standalone Python script designed to automatically process and visualize all simulation results generated by the UDP, TCP/IP, and QUIC experiments. It scans the **results/** directory, extracts final statistics (average packet delay, throughput, and packet loss ratio) from each protocol's CSV output, and organizes the data into a structured pandas' **DataFrame**. Using this dataset, the script generates three performance comparison graphs — *Delay vs. Number of UEs*, *Throughput vs. Payload Size*, and *Loss Ratio vs. Number of UEs* — enabling direct protocol-to-protocol evaluation. All graphs are saved as high-resolution PNG files in the **graphs/** folder, making the script a fully automated analysis tool for batch simulations and reproducible performance evaluation.
- Computed and plotted metrics across two main dimensions:
 - Number of UEs vs. Delay / Throughput / Loss Ratio
 - Payload Size vs. Throughput / Delay
- Automated export of all graphs to PNG/JPG/JPEG image format for report inclusion.

Outcome

High-quality visualizations that demonstrate protocol behavior and performance trends under varying UE densities and payload sizes.

Phase 7: Conduct Dual Connectivity and Handover Evaluation

Objective

Assess how dual connectivity and inter-gNB handover impact network performance.

[Type here]

Implementation

- Enabled handover procedures between gNB1 and gNB2.
- Verified UE re-association and continuity of session during handover events.
- Measured how throughput and latency evolve before, during, and after handover.
- Analyzed dual-connectivity capability (if enabled) to improve throughput and resilience.

Outcome

Quantitative and visual results that demonstrate the influence of handover and dual connectivity on transport-layer performance, validating 5G robustness across mobility scenarios.

User Manual: Installation and Testing Guide

This user manual provides a complete guide for installing, configuring, running, and validating the 5G Handover Analysis Simulation Environment across all major platforms: Windows, Linux, and macOS.

It ensures that any user—regardless of operating system—can successfully reproduce the simulation results.

System Requirements

Operating System

- Windows 11 (fully tested) / Windows 10 (compatible)
 - Users who use older versions are recommended to upgrade to Windows 10 or above.
- Ubuntu Linux 20.04 / 22.04
- macOS Monterey / Ventura / Sonoma

Hardware Requirements

- CPU: Dual-core or better (Quad-core recommended)
- RAM: Minimum 8GB (16GB recommended)
- Disk: 20GB free space
- Recommended: A device with at least 4 threads for faster compilation

Software Components

Component	Version	Required for
OMNeT++	6.2.0	Simulation engine
INET Framework	4.5.0	TCP/UDP/IP, mobility, routing stack

[Type here]

Simu5G	1.4.0	5G NR PHY/MAC/RLC/PDCP, gNB, UE, handover
Python 3.10+	–	Data analysis (CSV to graphs)
Pandas, Matplotlib (Python)	latest	Data visualization

Installation Instructions

Windows

Step 1: Install OMNeT++

1. Download OMNeT++ 6.2.0 (Windows version)
2. Extract into: `C:\omnetpp-6.2.0\`
3. Open the OMNeT++ Command Prompt and run:

```
1. ./configure
2. make
```

Step 2: Install INET

1. Extract INET into `C:\omnetpp-6.2.0\samples\inet4.5\`.
2. Import into OMNeT++: **File** → **Import** → **Existing Project**
3. Build: **Project** → **Build All**

Step 3: Install Simu5G

1. Extract Simu5G into `C:\omnetpp-6.2.0\samples\Simu5G-1.4.0\`.
2. Import and build.

Linux

Step 1: Install Build Dependencies

Run these commands:

```
1. sudo apt update
2. sudo apt install -y build-essential gcc g++ clang cmake \
3.     bison flex perl qtbase5-dev qtchooser qtscript5-dev \
4.     libqt5opengl5-dev libgl1-mesa-dev libglu1-mesa-dev \
5.     libxml2-dev zlib1g-dev
```

Step 2: Install OMNeT++

Run these commands:

```
1. tar -xvf omnetpp-6.2.0-src.tgz
2. cd omnetpp-6.2.0
```

[Type here]

```
3. ./configure
4. make -j$(nproc)
```

Step 3: Install INET & Simu5G

Run these commands:

```
1. cp -r inet4.5/ ~/omnetpp-6.2.0/samples/
2. cp -r Simu5G-1.4.0/ ~/omnetpp-6.2.0/samples/
```

Import into OMNeT++ or compile manually:

```
1. make -j$(nproc)
```

macOS

Step 1: Install Xcode CLI tools

```
1. xcode-select --install
```

Step 2: Install Homebrew & dependencies

```
1. brew install cmake openssl flex bison pkg-config autoconf automake libtool python3
```

Step 3: Build OMNeT++ on macOS

```
1. cd /Applications/omnetpp-6.2.0/
2. ./configure
3. make -j8
```

Step 4: Build INET & Simu5G

Place them inside:

```
1. /Applications/omnetpp-6.2.0/samples/
```

Then build using IDE or:

```
1. make -j8
```

Project Directory Setup

Download or clone the project into:

- **Windows:** C:\Users\<user>\OneDrive\Desktop\5G-Handover-Analysis\
- **Linux:** ~/Desktop/5G-Handover-Analysis/
- **macOS:** ~/Desktop/5G-Handover-Analysis/

Folder Structure

```
1. 5G-Handover-Analysis/
2. |
3. |— src/simu5g/apps/filetransfer_multicell/
4. |   |— FileSenderUDP.*, FileReceiverUDP.*
```

[Type here]

```
5. | FileSenderTCP.*, FileReceiverTCP.*
6. | FileSenderQUIC.*, FileReceiverQUIC.*
7. |
8. | simulations/nr/handover_sim/
9. | | MultiCell_Standalone.ned
10. | | omnetpp.ini
11. | | demo.xml
12. | | results/
13. | | graphs/
14. |
15. | analyze_results.py
16. |
```

Run the Simulation

Step 1: Open OMNeT++ IDE

Import your custom project into the OMNeT++ workspace.

Step 2: Clean & Build

Menu:

- Project → Clean
- Project → Build All

Step 3: Open omnetpp.ini

Choose one experiment:

1. Multicell-Handover-UDP
2. Multicell-Handover-TCP
3. Multicell-Handover-QUIC

Step 4: Run

Click the **Run** button.

- You can also click **Run As → 1 OMNeT++ Simulation**
- Or Run Configurations → <Your Project Name>
 - Choose **Config name**.
 - Enter run number (one or multiple **Run(s)**), for example: 0, 49, 99
 - Choose user interface: **Qtenv** or **Cmdenv**.
 - Run.

The simulator executes:

- UE mobility
- gNB handover
- Transport-layer transmission

[Type here]

- CSV logging

Expected Output

Files appear under:

- results/UDP/udp_runXX_ueYY_stats.csv
- results/TCP/tcp_runXX_ueYY_stats.csv
- results/QUIC/quic_runXX_ueYY_stats.csv

Each file contains:

- Delay
- Inter-packet gap
- Instantaneous throughput
- Packet loss ratio
- Final summary block.

Step 5: Running the Python Analysis Script

1. Install Python dependencies:

For Windows/macOS/Linux:

```
1. pip install pandas matplotlib
```

2. Execute the script:

```
1. cd 5G-Handover-Analysis
2. python analyze_results.py
```

Output

Inside **graphs/**:

- delay_vs_ues.png
- throughput_vs_payload.png
- loss_vs_ues.png

Cross-Platform Troubleshooting Guide

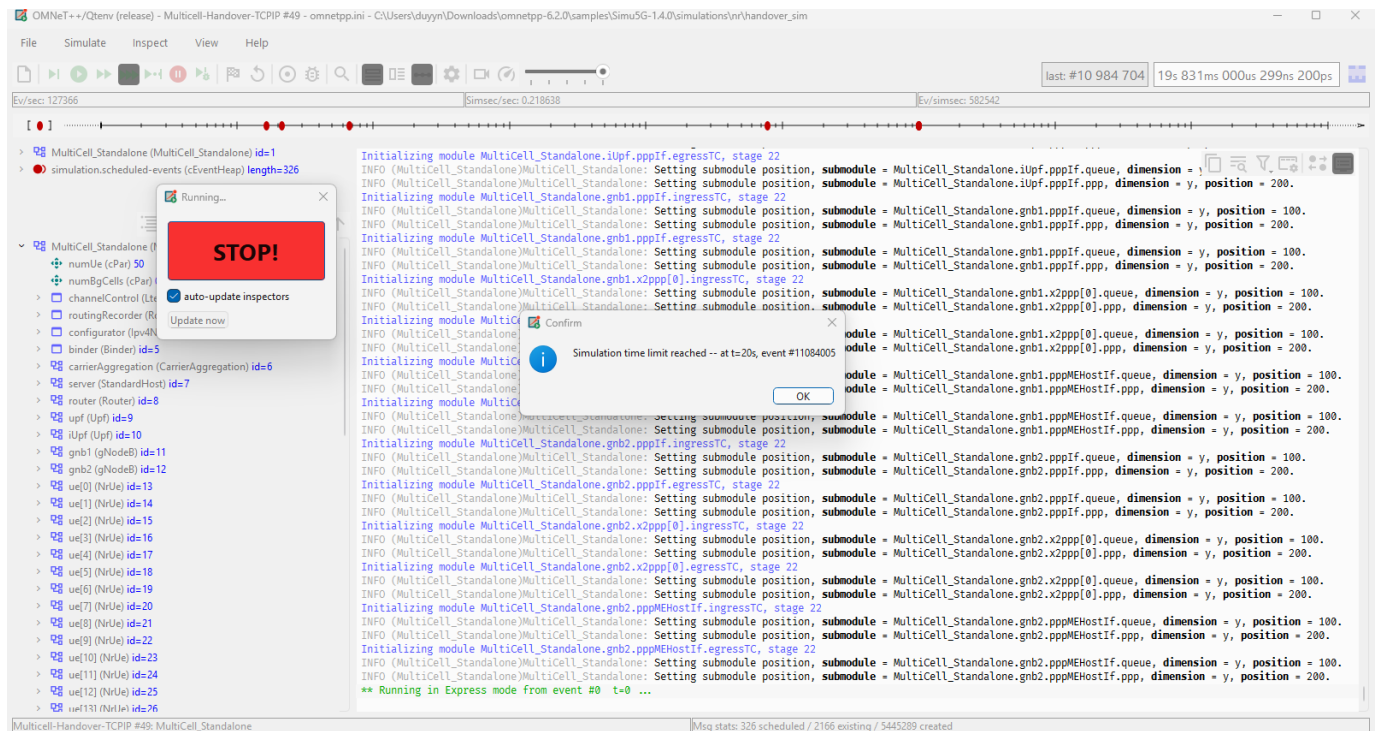
Issue	Windows fix	Linux fix	macOS fix
Class not found	Rebuild project	Run these commands: make clean && make	Ensure namespaces correct
Header not found	Refresh workspace	Install dev packages	Fix include paths

[Type here]

Blank graphs	CSV missing	Permission issues	Ensure UTF-8 BOM
No simulation output	Start time < warmup period	Increase warmup	Same fix
Simu5G build errors	Use MinGW-w64	Install bison/flex	Install flex/bison via Homebrew

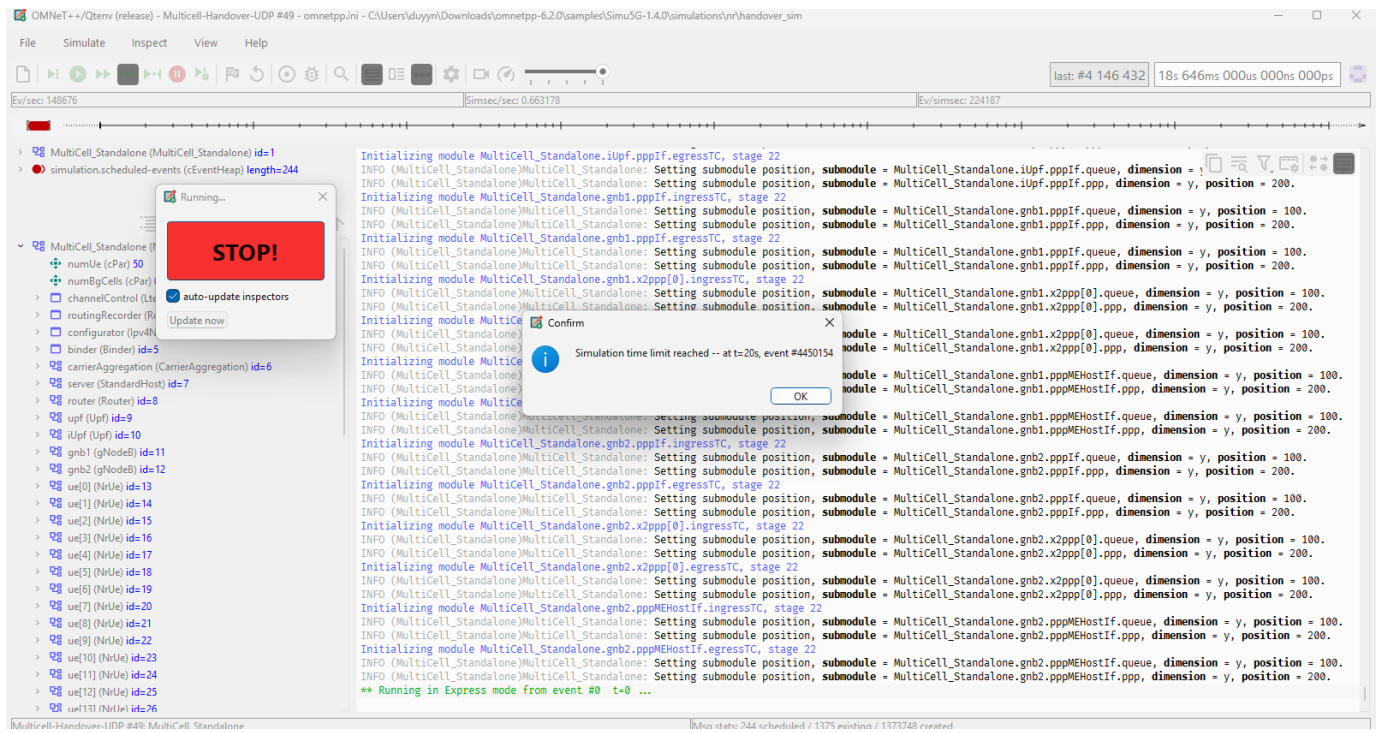
Results/Conclusion

Simulation Running

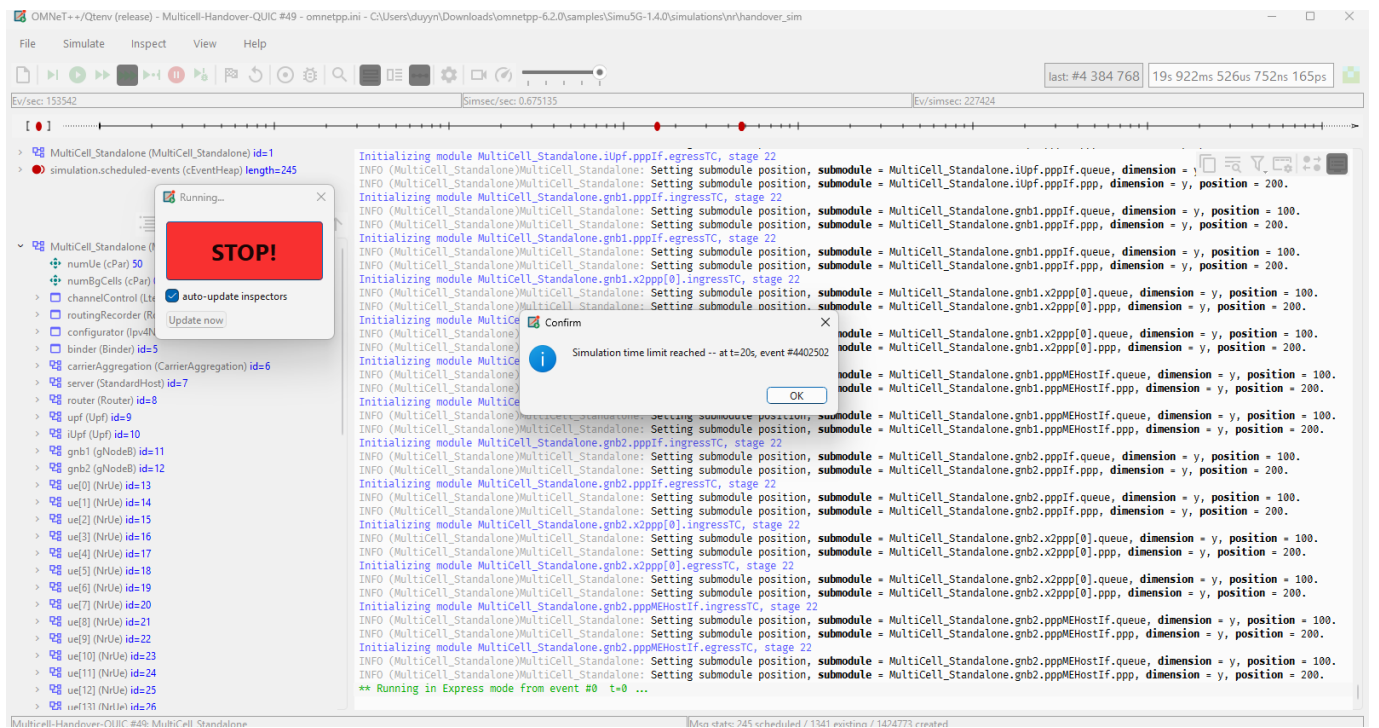


1. Simulation running for TCP/IP protocol (run #49)

[Type here]



2. Simulation running for UDP protocol (run #49)



3. Simulation running for QUIC protocol, run #49

[Type here]

Performance Analysis

940	*** Final Statistics 최종 통계 ***			
941	Frames Received(Current/Total) 수신된 프레임(현재/전체): 937/937			
942	Average Packet Delay 평균 패킷 지연: 0.00440258초 seconds			
943	Average Inter-Packet Gap 평균 패킷 간격: 0.0213383초 seconds			
944	Throughput 처리량: 0.394526 Mbps			
945	Packet Loss Ratio 패킷 손실률: 0%			
946				
< > quic_run49_ue49_stats +				

4. Statistics for QUIC protocol (run #49)

1884	*** Final Statistics 최종 통계 ***			
1885	Frames Received(Current/Total) 수신된 프레임(현재/전체): 1881/1882			
1886	Average Packet Delay 평균 패킷 지연: 0.004초 seconds			
1887	Average Inter-Packet Gap 평균 패킷 간격: 0.0106273초 seconds			
1888	Throughput 처리량: 0.424512 Mbps			
1889	Packet Loss Ratio 패킷 손실률: 0%			
1890				
< > tcp_run49_ue49_stats +				

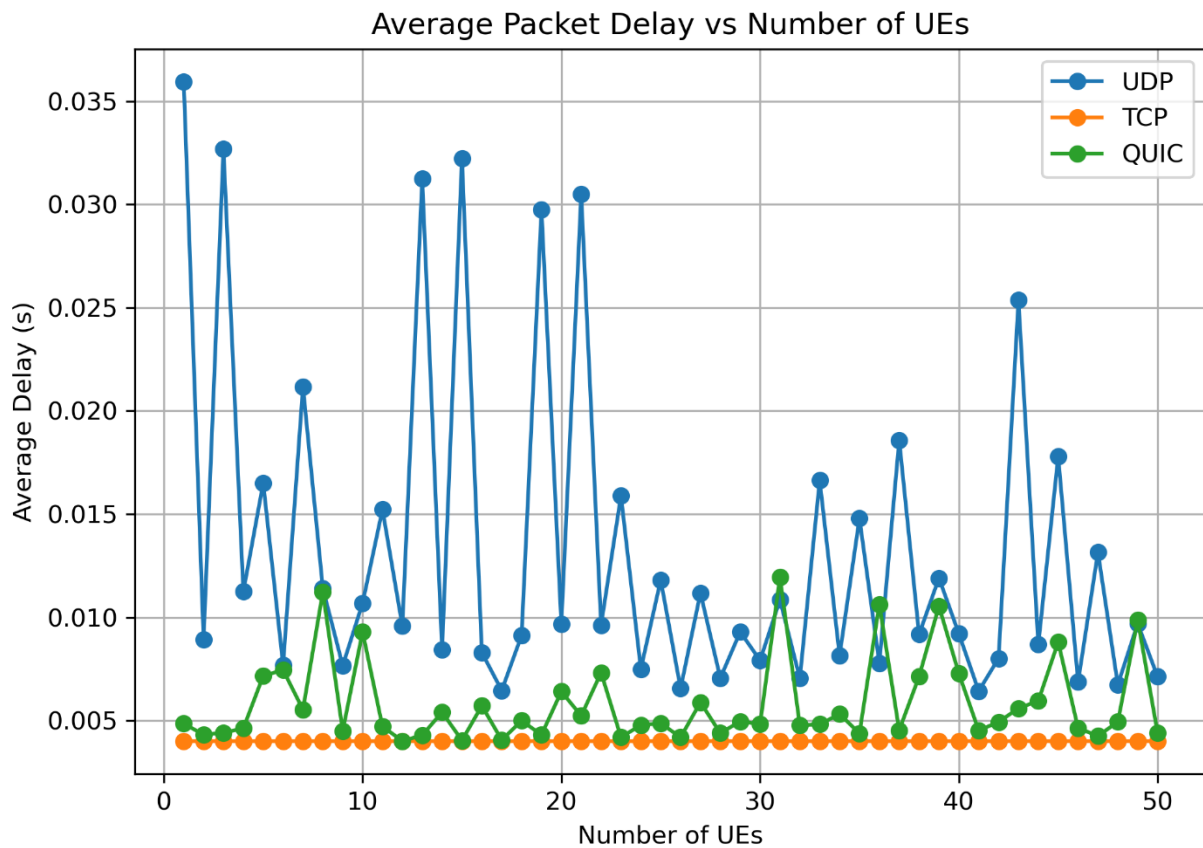
5. Statistics for TCP/IP protocol, run #49

936	*** Final Statistics 최종 통계 ***			
937	Frames Received(Current/Total) 수신된 프레임(현재/전체): 933/942			
938	Average Packet Delay 평균 패킷 지연: 0.00712176초 seconds			
939	Average Inter-Packet Gap 평균 패킷 간격: 0.0214298초 seconds			
940	Throughput 처리량: 0.392842 Mbps			
941	Packet Loss Ratio 패킷 손실률: 0.955414%			
942				
< > udp_run49_ue49_stats +				

6. Statistics for UDP protocol (run #49)

[Type here]

Graphs & Summary



UDP:

- Delay is highly unstable and spikes frequently.
- Delay ranges: 0.007 – 0.035 s.
- UDP has no congestion control, so delay increases sharply with more UEs and interference.
- When queues overflow, UDP suffers more, causing the sharp peaks.

TCP/IP:

- Delay is very stable and lowest.
- Almost a baseline around 0.0038–0.0042 s.
- TCP/IP adapts through congestion control and retransmissions, maintaining stable latency.

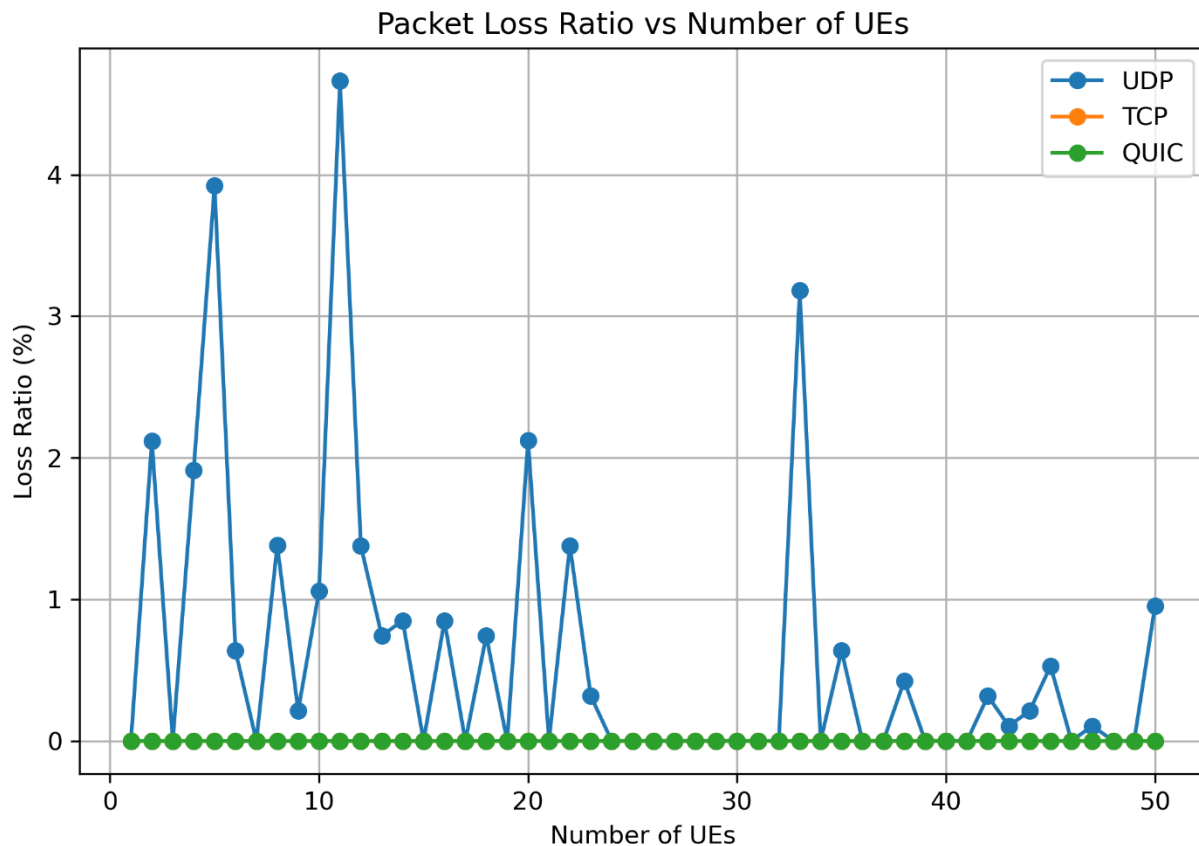
QUIC:

- Delay is slightly higher than TCP/IP but much more stable than UDP.

[Type here]

- Typical values: 0.0045–0.0065 s, with occasional spikes.
- QUIC's congestion control and multiplexing stabilize performance, but not as aggressively as TCP/IP.

Conclusion (Delay Ranking): TCP/IP (best) > QUIC > UDP (worst)



UDP:

- Loss ratio spikes up to:
 - 2–4% at 10 UEs
 - Still some loss around 1% near 20–40 UEs
- UDP cannot retransmit, so any congestion or handover loss is permanent.

TCP:

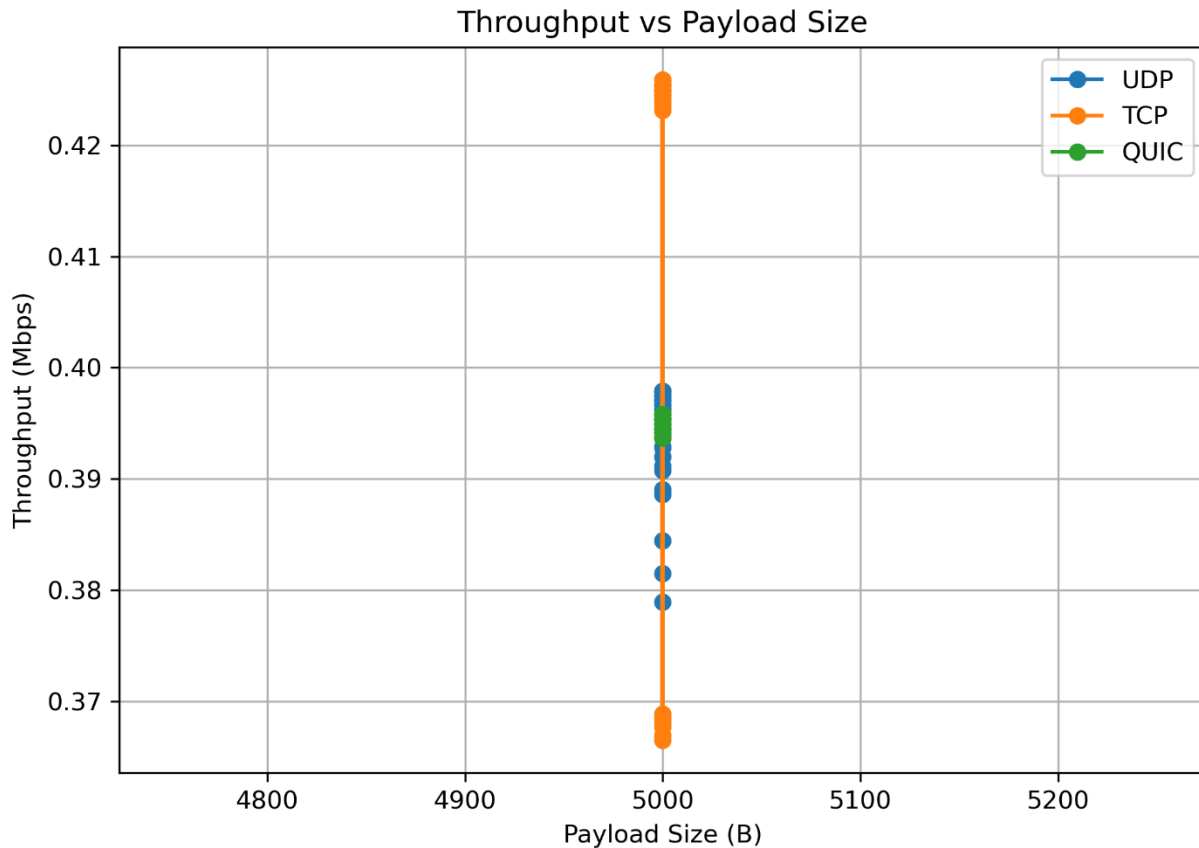
- 0% loss, always.
- All losses are recovered via retransmission.
- Expected behaviour—TCP/IP faithfully delivers.

QUIC:

[Type here]

- 0% loss reported.
- QUIC also retransmits lost packets at the QUIC level (not UDP).

Conclusion (Loss Ranking): TCP/IP = QUIC (0% loss) > UDP (worst)



Observation

- Throughput clusters around:
 - **UDP:** ~0.38–0.395 Mbps
 - **TCP:** ~0.425 Mbps (highest)
 - **QUIC:** ~0.39–0.395 Mbps (slightly below UDP and far below TCP)

Interpretation

- TCP/IP achieves the highest throughput because of window scaling, congestion control, and guaranteed delivery.
- UDP $\approx$ QUIC, but QUIC is slightly better because it avoids kernel slow paths and optimizes ACK behaviour.
- QUIC still depends on UDP, so it has overhead during congestion.

[Type here]

Conclusion (Throughput Ranking): TCP/IP (highest) > QUIC ≥ UDP

Performance Comparison

This study evaluated UDP, TCP/IP, and QUIC under 5G multi-cell mobility using Simu5G. Three key metrics were examined: latency, packet loss, and throughput, under varying UE loads and payload sizes.

Latency

TCP/IP provided the **lowest and most stable** latency (~4 ms), benefiting from congestion control and flow regulation. QUIC performed slightly worse (~4–6 ms) but remained stable. UDP experienced **significant latency spikes** (up to 30–35 ms), especially as UE count increased due to lack of congestion handling.

Packet Loss

TCP/IP and QUIC achieved **0% effective loss**, fully recovering dropped segments. UDP, by design, performed worst, with loss rates fluctuating from **0.5% to 4%**, especially during handover events or congestion.

Throughput

TCP/IP achieved the **highest throughput** (~0.425 Mbps). QUIC and UDP achieved similar throughput (~0.39 Mbps), with QUIC slightly outperforming UDP due to better flow control but falling short of TCP/IP due to its encryption and congestion overhead.